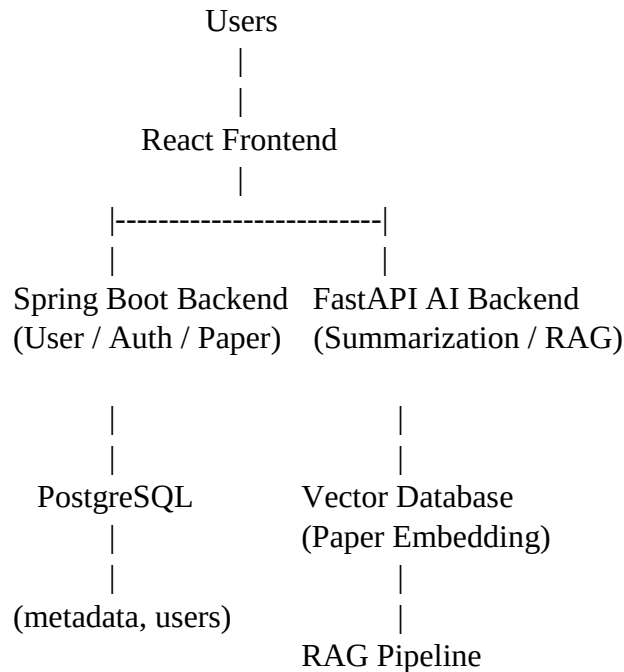


# Summarize papers

## 1. Kiến trúc tổng thể của hệ thống



### Vai trò từng phần

#### - Người dùng(Users):

Người dùng truy cập hệ thống để:

- Upload **paper PDF**
- Nhập **đoạn văn hoặc link paper**
- Yêu cầu **tóm tắt nội dung**
- Tìm kiếm và xem lại lịch sử

#### - React Frontend:

Đây là **giao diện người dùng (UI)**.

Chức năng chính:

- Form upload PDF
- Hiển thị bản tóm tắt
- Tìm kiếm paper
- Xem lịch sử
- Gửi request đến backend (Spring Boot + FastAPI)

#### - Spring Boot Backend:

Spring Boot đóng vai trò **core backend**:

API User, API User, ...

### **Chức năng:**

- Quản lý user (authentication, authorization – JWT)
- Lưu metadata paper
- Quản lý file PDF ...

Database: PostgreSQL

Lưu: Users, Papers, Metadata, History, ...

### **- FastAPI (AI backend)**

Đây là **backend AI của hệ thống**.

Nhiệm vụ:

- xử lý NLP
- chạy model tóm tắt
- thực hiện pipeline RAG

### **Vector Database:**

Đây là **database lưu embedding của paper**.

Embedding = vector biểu diễn nội dung văn bản.

Ví dụ:

paper chunk → embedding vector

Các công nghệ thường dùng:

FAISS

Chroma

Weaviate

Mục đích:

- tìm các đoạn văn **liên quan nhất**
- phục vụ RAG.

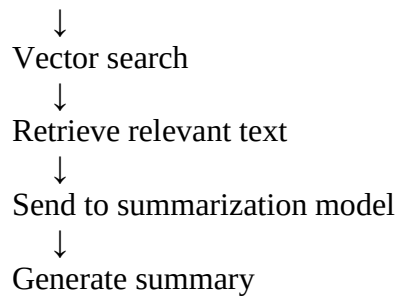
### **RAG Pipeline:**

Quy trình:

User request



Embedding query



Ví dụ:

Paper: 20 trang  
↓  
chia thành 50 đoạn  
↓  
tìm đoạn quan trọng  
↓  
tóm tắt

## 2. Chuẩn bị dataset (DataOps)

### Dataset sử dụng

Để huấn luyện mô hình tóm tắt, hệ thống cần dữ liệu gồm **văn bản gốc và bản tóm tắt tương ứng**.

Các nguồn dữ liệu có thể sử dụng:

- Vietnamese news summarization datasets
- VnExpress dataset (article – summary)
- Vietnamese academic papers (sử dụng abstract làm summary)

Dạng dữ liệu:

Paper Text → Abstract

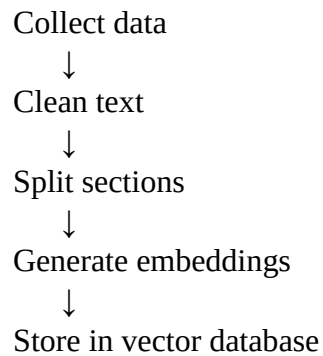
Trong đó:

- **Paper Text**: nội dung bài báo
- **Abstract**: phần tóm tắt chuẩn của bài báo

Dataset sau khi xử lý sẽ có dạng:

```
{  
  "document": "...paper content...",  
  "summary": "...paper abstract..."  
}
```

- **Pipeline dữ liệu:**



### Script cần viết

#### 1 PDF parser

pdf -> text

#### 2 Text cleaner

remove references  
remove figure  
normalize text

#### 3 Chunking

paper -> đoạn 500 tokens

#### 4 Embedding

sentence-transformers

### 3. Xây dựng mô hình Summarization

Trong giai đoạn này, hệ thống sẽ xây dựng mô hình **tóm tắt tự động cho các bài báo khoa học**. Thay vì huấn luyện mô hình từ đầu, phương pháp hiệu quả hơn là **fine-tune các mô hình đã được huấn luyện sẵn (pre-trained models)** trên dữ liệu paper.

#### 3.1. Các mô hình có thể sử dụng

Một số mô hình phổ biến cho bài toán **abstractive summarization** gồm:

- **BART**
- **T5**
- **LongT5**
- **PEGASUS**

Đối với dữ liệu **tiếng Việt**, có thể sử dụng các mô hình được huấn luyện riêng cho ngôn ngữ này như:

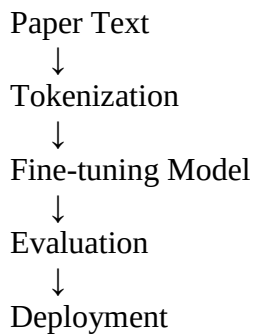
- **ViT5**

- **BARTpho**

Các mô hình này dựa trên kiến trúc **Transformer encoder-decoder**, rất phù hợp cho các bài toán sinh văn bản như tóm tắt.

### 3.2. Pipeline huấn luyện mô hình

Quy trình huấn luyện mô hình tóm tắt gồm các bước sau:



Giải thích các bước:

1. **Thu thập và chuẩn hóa dữ liệu**
  - Trích xuất nội dung văn bản từ file PDF của các bài báo.
  - Làm sạch dữ liệu (loại bỏ hình, bảng, references không cần thiết).
2. **Tokenization**
  - Chuyển văn bản thành các token để mô hình có thể xử lý.
3. **Fine-tuning**
  - Huấn luyện lại mô hình pre-trained trên tập dữ liệu paper để tối ưu cho nhiệm vụ tóm tắt.
4. **Evaluation**
  - Đánh giá chất lượng bản tóm tắt do mô hình tạo ra.
5. **Deployment**
  - Triển khai mô hình dưới dạng API thông qua backend AI (FastAPI).

---

### 3.3. Metrics đánh giá mô hình

Chất lượng của mô hình tóm tắt được đánh giá bằng các chỉ số phổ biến trong bài toán summarization:

- **ROUGE-1**: đo mức độ trùng khớp unigram giữa summary sinh ra và summary chuẩn.
- **ROUGE-2**: đo mức độ trùng khớp bigram.
- **ROUGE-L**: đo độ tương đồng dựa trên chuỗi con dài nhất (Longest Common Subsequence).

Các chỉ số **ROUGE** càng cao thì bản tóm tắt càng gần với bản tóm tắt chuẩn.

## 4. Xây dựng RAG Pipeline

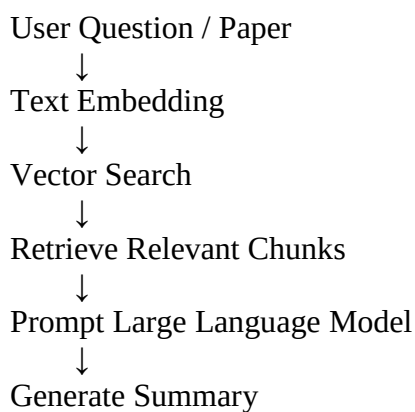
Trong hệ thống tóm tắt bài báo khoa học, nhóm áp dụng phương pháp **Retrieval-Augmented Generation (RAG)** để cải thiện chất lượng tóm tắt.

RAG kết hợp **truy xuất thông tin (retrieval)** với **mô hình sinh văn bản (generation)**. Thay vì để mô hình sinh văn bản chỉ dựa trên kiến thức đã học, hệ thống sẽ **truy xuất các đoạn văn bản liên quan từ cơ sở dữ liệu** và sử dụng chúng làm ngữ cảnh cho mô hình tạo ra bản tóm tắt chính xác hơn.

---

#### 4.1. Pipeline của hệ thống RAG

Quy trình xử lý của hệ thống RAG được thực hiện theo các bước sau:



Giải thích các bước:

1. **Input**
    - Người dùng cung cấp nội dung bài báo hoặc yêu cầu tóm tắt.
  2. **Embedding**
    - Văn bản được chuyển thành vector embedding để biểu diễn ngữ nghĩa.
  3. **Vector Search**
    - Hệ thống tìm kiếm trong vector database để xác định các đoạn văn bản liên quan nhất.
  4. **Retrieve Chunks**
    - Các đoạn văn bản phù hợp được trích xuất làm ngữ cảnh cho mô hình.
  5. **Prompt LLM**
    - Các đoạn văn bản được đưa vào prompt của mô hình ngôn ngữ lớn.
  6. **Generate Summary**
    - Mô hình sinh ra bản tóm tắt dựa trên ngữ cảnh đã truy xuất.
- 

#### 4.2. Công nghệ sử dụng

Hệ thống RAG được triển khai bằng các công nghệ sau:

## 1. *Embedding Model*

Embedding được sử dụng để chuyển văn bản thành vector ngữ nghĩa.

Ví dụ:

- thư viện **Sentence-Transformers**
- 

## 2. *Vector Database*

Vector database được sử dụng để lưu trữ và tìm kiếm các embedding của văn bản.

Một số công nghệ phổ biến:

- **FAISS**
- **Chroma**
- **Weaviate**

Các hệ thống này hỗ trợ **tìm kiếm ngữ nghĩa (semantic search)** nhanh trên dữ liệu văn bản lớn.

---

## 3. *Large Language Model (LLM)*

Sau khi truy xuất được các đoạn văn bản liên quan, hệ thống sẽ sử dụng mô hình ngôn ngữ lớn để tạo bản tóm tắt.

Một số mô hình có thể sử dụng:

- **LLaMA**
  - **\*\*OpenAI models**
  - **Mistral**
- 

### 4.3. Ví dụ Prompt cho hệ thống

Một ví dụ prompt được sử dụng để yêu cầu mô hình tạo bản tóm tắt:

Summarize the following research paper.

Context:

{retrieved\_chunks}

Provide a structured summary including:

1. Objective
2. Method

3. Results

4. Contribution

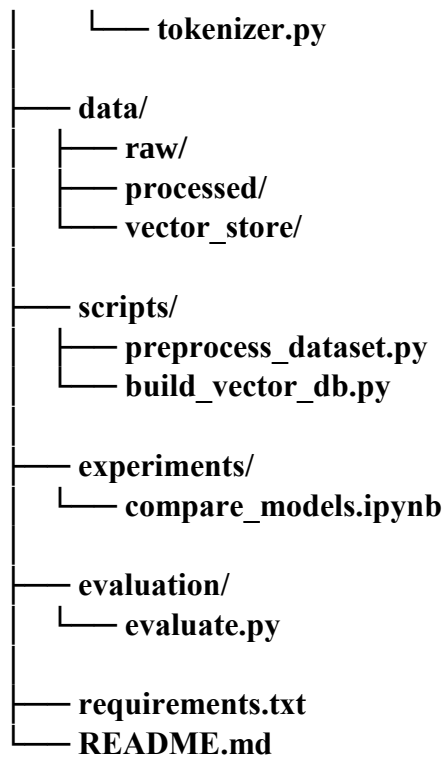
Prompt này giúp mô hình tạo ra bản tóm tắt **có cấu trúc rõ ràng**, phù hợp với nội dung của một bài báo khoa học.

## 5. Backend AI (FastAPI)

Cấu trúc thư mục gợi ý cho FastAPI AI backend:

```
ai-service-fastapi/  
├── app/  
│   ├── main.py  
│   ├── api/  
│   │   ├── summarize.py  
│   │   ├── ask.py  
│   │   └── paper.py  
│   ├── services/  
│   │   ├── summarization_service.py  
│   │   ├── embedding_service.py  
│   │   └── rag_service.py  
│   ├── models/  
│   │   ├── summarizer_model.py  
│   │   ├── embedding_model.py  
│   │   └── llm_model.py  
│   ├── rag/  
│   │   ├── pipeline.py  
│   │   ├── retriever.py  
│   │   ├── chunking.py  
│   │   └── prompt_template.py  
│   ├── vector_store/  
│   │   └── faiss_store.py  
│   ├── schemas/  
│   │   ├── request.py  
│   │   └── response.py  
│   └── utils/  
│       ├── pdf_parser.py  
│       └── text_cleaner.py
```





**Mô tả nhiệm vụ từng thư mục:**

[main.py](#)

Entry point của server.

[api/](#)

Chứa các **API endpoint**.

[services/](#)

Chứa **business logic** của AI system.

[models/](#)

Load các **AI model**.

Các model được load khi server start.

[rag/](#)

Chứa **RAG pipeline**.

Các thành phần:

- chunk text
- retrieve context

- build prompt
- gọi LLM

[vector\\_store/](#)

Xử lý **vector database**.

[schemas/](#)

Chứa **Pydantic models**.

Dùng để định nghĩa:

- request body
- response format

[utils/](#)

Các công cụ hỗ trợ:

- parse PDF
- clean text
- tokenize

[data/](#)

Chứa **dữ liệu của hệ thống AI**.

**raw/**

Dữ liệu gốc.

**processed/**

Text sau khi:

PDF → text  
clean → chunk

**vector\_store/**

Lưu:

embeddings  
FAISS index

**vector\_store/**

Lưu:

embeddings  
FAISS index

[experiments/](#)

Notebook để thử nghiệm mô hình.

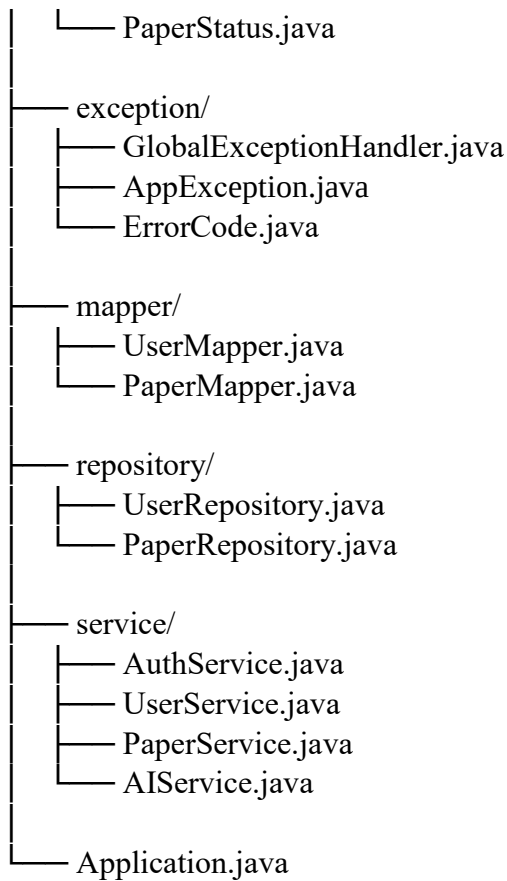
[evaluation/](#)

Script đánh giá model.

## 6. Backend Spring Boot

Cấu trúc thư mục ( đề xuất):

```
springboot-backend/  
├── configuration/  
│   └── security/  
│       ├── JwtAuthenticationFilter.java  
│       ├── JwtService.java  
│       └── SecurityConfig.java  
├── controller/  
│   ├── AuthController.java  
│   ├── UserController.java  
│   └── PaperController.java  
├── dto/  
│   ├── request/  
│   │   ├── LoginRequest.java  
│   │   ├── RegisterRequest.java  
│   │   ├── UploadPaperRequest.java  
│   │   └── SummarizeRequest.java  
│   └── response/  
│       ├── AuthResponse.java  
│       ├── UserResponse.java  
│       ├── PaperResponse.java  
│       └── SummaryResponse.java  
├── entity/  
│   ├── User.java  
│   └── Paper.java  
└── enums/  
    └── Role.java
```



### Mô tả nhiệm vụ từng thư mục:

#### [1. configuration/](#)

Thư mục này chứa các cấu hình của hệ thống.

#### Nhiệm vụ:

- Cấu hình **Spring Security**
- Xử lý **JWT Authentication**
- Định nghĩa các filter bảo mật

#### Các thành phần chính:

- SecurityConfig: cấu hình quyền truy cập API
- JwtAuthenticationFilter: kiểm tra token mỗi request
- JwtService: tạo và validate JWT

---

#### [2. controller/](#)

Đây là tầng **API (Presentation Layer)** của hệ thống.

### Nhiệm vụ:

- Nhận request từ frontend (React)
- Gọi service để xử lý logic
- Trả response về client

### Ví dụ:

- AuthController: đăng ký, đăng nhập
- UserController: thông tin user
- PaperController: upload, lấy danh sách, tóm tắt

---

### 3. dto/

DTO (Data Transfer Object) dùng để **truyền dữ liệu giữa client ↔ server**

---

#### **dto/request/**

Chứa các object nhận dữ liệu từ client

### Ví dụ:

- LoginRequest
- RegisterRequest
- UploadPaperRequest
- SummarizeRequest

☞ Dùng để:

- validate input
  - tránh expose entity trực tiếp
- 

#### **dto/response/**

Chứa dữ liệu trả về cho client

### Ví dụ:

- AuthResponse
  - UserResponse
  - PaperResponse
  - SummaryResponse
-

#### 4. entity/

Đây là tầng **mapping database**

**Nhiệm vụ:**

- Định nghĩa bảng trong PostgreSQL
- Mapping với JPA/Hibernate

**Ví dụ:**

- User
  - Paper
- 

#### 5. enums/

Chứa các kiểu dữ liệu enum dùng trong hệ thống

**Nhiệm vụ:**

- Chuẩn hóa giá trị cố định
- Tránh hardcode string

**Ví dụ:**

- Role (ADMIN, USER)
  - PaperStatus (UPLOADED, PROCESSING, DONE)
- 

#### 6. exception/

Xử lý lỗi toàn hệ thống

**Nhiệm vụ:**

- Custom exception
- Handle lỗi tập trung
- Trả lỗi chuẩn REST API

**Thành phần:**

- AppException: exception custom
  - ErrorCode: định nghĩa mã lỗi
  - GlobalExceptionHandler: bắt lỗi toàn cục
-

## 7. mapper/

Dùng để chuyển đổi dữ liệu giữa các layer

### Nhiệm vụ:

- Entity → Response DTO
- Request DTO → Entity

### Ví dụ:

- UserMapper
- PaperMapper

☞ Giúp code:

- sạch hơn
  - tách biệt logic
- 

## 8. repository/

Tầng làm việc với database

### Nhiệm vụ:

- CRUD dữ liệu
- Query database

### Công nghệ:

- Spring Data JPA

### Ví dụ:

- UserRepository
  - PaperRepository
- 

## 9. service/

Đây là **tầng business logic (quan trọng nhất)**

---

### Nhiệm vụ chung:

- Xử lý logic nghiệp vụ
- Kết nối giữa controller và repository

- Gọi AI backend (FastAPI)
- 

### **Các service chính:**

#### *AuthService*

- Đăng ký
  - Đăng nhập
  - Tạo JWT
- 

#### *UserService*

- Lấy thông tin user
  - Update profile
- 

#### *PaperService*

- Upload paper
- Lưu DB
- Quản lý paper
- Gọi AI để tóm tắt